



MySQL - RDBMS

Trainer: Nisha Dingare



MySQL data types Continued : Binary types

- **Storage Size**
- The size of binary data is the **number of bytes** stored.

Data Type	Maximum Length (bytes)	Description
BINARY(n)	Fixed length, up to 255 bytes	Stores fixed-length binary data
VARBINARY(n)	Variable length, up to 65,535 bytes	Stores variable-length binary data
TINYBLOB	255 bytes	Small Binary Large Object
BLOB BLOB = Binary Large Object a container for raw binary files inside a database.	65,535 bytes (~64 KB)	Standard Binary Large Object
MEDIUMBLOB	16,777,215 bytes (~16 MB)	Medium-sized Binary Large Object
LOBBLOB	4,294,967,295 bytes (~4 GB)	Large Binary Large Object

- Binary types are used to store raw binary data like images, audio, video, or any other non-text data.
- Sizes can vary widely from small to very large objects depending on the chosen type.



MySQL data types : Miscellaneous Data Types

Data Type	Description	Usage / Notes
ENUM	Enumeration: a column that stores one value from a predefined list	Acts like a radio button — only one value can be selected from the list of allowed values. Efficient storage, stored internally as integers
SET	Set: a column that stores zero or more values from a predefined list	Acts like checkboxes — multiple values can be selected simultaneously. Also stored as integers internally for efficient storage
BOOLEAN / BOOL	Stores truth values, TRUE or FALSE	Internally treated as TINYINT(1), where 0 = FALSE, 1 = TRUE



INSERT – DML

- Insert a new row (all columns, fixed order).
 - `INSERT INTO table VALUES (v1, v2, v3);`
- Insert a new row (specific columns, arbitrary order).
 - `INSERT INTO table(c3, c1, c2) VALUES (v3, v1, v2);`
 - `INSERT INTO table(c1, c2) VALUES (v1, v2);`
 - Missing columns data is NULL.
 - NULL is special value and it is not stored in database.
- Insert multiple rows.
 - `INSERT INTO table VALUES (av1, av2, av3), (bv1, bv2, bv3), (cv1, cv2, cv3).`
- Insert rows from another table.
 - `INSERT INTO table SELECT c1, c2, c3 FROM another-table;`
 - `INSERT INTO table (c1,c2) SELECT c1, c2 FROM another-table;`



SQL scripts

- SQL script is multiple SQL queries written into a .sql file.
- SQL scripts are mainly used while database backup and restore operations.
- SQL scripts can be executed from terminal as:
 - terminal> mysql -u user -ppassword db < /path/to/sqlfile
- SQL scripts can be executed from command line as:
 - mysql> SOURCE /path/to/sqlfile
- Note that SOURCE is MySQL CLI client command.
- It reads commands one by one from the script and execute them on server.



SELECT – DQL

- Select all columns (in fixed order).
 - `SELECT * FROM table;`
- Select specific columns / in arbitrary order.
 - `SELECT c1, c2, c3 FROM table;`
- **Column alias**
 - `SELECT c1 AS col1, c2 col2 FROM table;`
- Computed columns. *Check day2.sql for more detailed with examples*
 - `SELECT c1, c2, c3, expr1, expr2 FROM table;`
`SELECT c1,`
`CASE WHEN condition1 THEN value1,`
`WHEN condition2 THEN value2,`
`...`
`ELSE valuen`
`END`
`FROM table;`



SELECT – DQL WITH DISTINCT AND LIMIT

- **DISTINCT** : Get Distinct Values
- It helps remove duplicates.
- Syntax:
- **SELECT DISTINCT column_name FROM table_name;**
- Example:
- **SELECT DISTINCT deptno FROM emp;**
- Returns unique values from column deptno.

- **Multiple Columns:**
- **SELECT DISTINCT deptno, job FROM emp;**
- Returns unique combinations of values from both columns.

- **LIMIT** : Limit the Number of Rows
- IT helps control the number of records displayed.
- Syntax:
- **SELECT * FROM table LIMIT n;**
- Returns the first n rows from the table.
- Example:
- **SELECT * FROM emp LIMIT 5;**

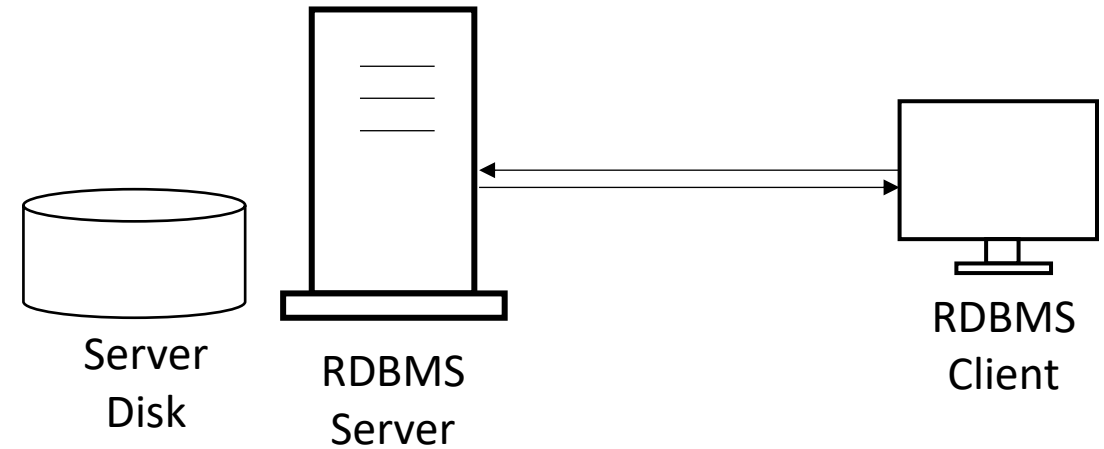
- **Limit with Offset**
- Syntax:
- **SELECT * FROM table LIMIT m, n;**
- Skips the first m rows and returns the next n rows.

- Example:
- **SELECT * FROM emp LIMIT 5, 10;**
- Skips 5 rows, then retrieves 10 rows.



SELECT – DQL – ORDER BY

- In db rows are scattered on disk. Hence may not be fetched in a fixed order.
- Select rows in asc order.
 - `SELECT * FROM table ORDER BY c1;`
 - `SELECT * FROM table ORDER BY c2 ASC;`
- Select rows in desc order.
 - `SELECT * FROM table ORDER BY c3 DESC;`
- Select rows sorted on multiple columns.
 - `SELECT * FROM table ORDER BY c1, c2;`
 - `SELECT * FROM table ORDER BY c1 ASC, c2 DESC;`
 - `SELECT * FROM table ORDER BY c1 DESC, c2 DESC;`
- Select top or bottom n rows.
 - `SELECT * FROM table ORDER BY c1 ASC LIMIT n;`
 - `SELECT * FROM table ORDER BY c1 DESC LIMIT n;`
 - `SELECT * FROM table ORDER BY c1 ASC LIMIT m, n;`



SELECT – DQL – WHERE

- It is always good idea to fetch only required rows (to reduce network traffic).
- The WHERE clause is used to specify the condition, which records to be fetched.
- Relational operators
 - <, >, <=, >=, =, != or <>
- NULL related operators
 - NULL is special value and cannot be compared using relational operators.
 - IS NULL or <=>, IS NOT NULL.
- Logical operators
 - AND, OR, NOT



SELECT – DQL – WHERE

- **BETWEEN operator (include both ends)** Check more in day2.sql file
 - **c1 BETWEEN val1 AND val2**

```
SELECT *
FROM Student
WHERE marks BETWEEN 50 AND 80;
```
- **IN operator (equality check with multiple values)**
 - c1 IN (val1, val2, val3)

```
SELECT *
FROM Salespeople
WHERE city IN ('London', 'San Jose');
```

IN is same as = like
city = 'London' AND
city = 'San Jose';
- **LIKE operator (similar strings)**
 - c1 LIKE 'pattern'.
 - % represent any number of any characters.
 - _ represent any single character.

```
SELECT *
FROM Salespeople
WHERE city NOT IN ('London', 'San Jose');
```

LIKE — Most Used Patterns				
Pattern	Meaning	Example	Matches	
'P%'	Starts with P	name LIKE 'P%'	Peter, Pune, Priya	
'%a'	Ends with a	name LIKE '%a'	Priya, Pooja, Riya	
'%a%'	Contains a anywhere	name LIKE '%a%'	Rahul, Anil, Maya	
'_a%'	Second character is a	name LIKE '_a%'	Rahul, Patel	
'P_'	Starts with P + exactly 1 more character	name LIKE 'P_'	Pa, Pi, Po	
'____'	Exactly 4 characters	name LIKE '____'	Riya, John, Peel	

THANK YOU

